Ch 05: Local and Global Variables, Arguments to Functions

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Local Variables

- **Local variable**: variable that is assigned a value inside of a function
  - Belongs to the function in which it was created
    - Only statements inside that function can access it, error will occur if another function tries to access the variable
- **Scope**: the part of a program in which a variable may be accessed
  - For local variable: function in which created is its scope
- Local variable cannot be accessed by statements inside its function which precede its creation (same with any variable, can't use until defined)
- Different functions may have local variables with the same name
  - Each function does not see the other function's local variables, so no confusion

Local Variables

A local variable is created inside a function and cannot be accessed by statements that are outside the function. Different functions can have local variables with the same names because the functions cannot see each other's local variables.



Passing Arguments to Functions (1 of 2)

Function Arguments

An argument is any piece of data that is passed into a function when the the function is called. A parameter is a variable that receives an argument that is passed into a function.

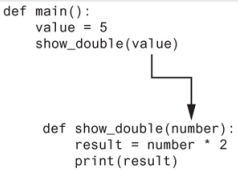


Figure 1: The value variable is passed as an argument (Gaddis, 2021, pg.259)

- **Argument**: piece of data that is sent into a function
  - Function can use argument in calculations
  - When calling the function, the argument is placed in parentheses following the function name
  - Argument is a local variable, local in scope to function, that is initialized with value that is passed in when function is called

EAST TEXAS A&M

Passing Arguments to Functions (2 of 2)

- **Parameter variable**: variable that is assigned the value of an argument when the function is called
  - The parameter references a copy of the value passed in to it
  - **Scope of a parameter**: the function in which the parameter is used
    - A parameter is a local variable that is initialized when the function is called by the input

```
def main():
    value = 5
    show_double(value)

def show_double(number):
    result = number * 2
    print(result)
```

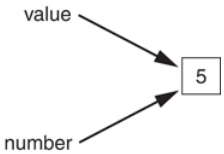


Figure 2: The value variable and the number parameter reference the same value (Gaddis, 2021, pg.260)

EAST TEXAS A&M

Passing Multiple Arguments

- Python allows writing a function that accepts multiple input arguments
  - Parameter list replaces single parameter
  - Parameter list items separated by comma
- Arguments are passed **by position** to corresponding parameters
  - First parameter receives value of first argument, second parameter gets value of second argument, etc.
  - Position matters (for positional arguments)

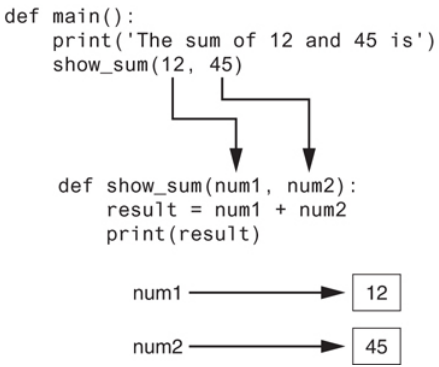


Figure 3: Two arguments passed to two parameters (Gaddis, 2021, pg.263)

EAST TEXAS A&M

Making Changes to Parameters (1 of 2)

Pass by Value

Changes made to a parameter value within the function do not affect the argument on return to the caller

```
def main():
    value = 99
    print(f'The value is {value}.')
    change_me(value)
    print(f'Back in main the value is {value}.')


def change_me(arg):
    print('I am changing the value.')
    arg = 0
    print(f'Now the value is {arg}.')
```

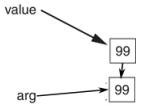


Figure 4: The value variable is passed to the change_me() function (Gaddis, 2021, pg.265)

- Known as **pass by value**
- Provides a way for unidirectional communication between one function and another function
  - Calling function can communicate with called function (not vice-versa)

EAST TEXAS A&M

Making Changes to Parameters (2 of 2)

- The `value` variable passed to the `change_me()` function cannot be changed by it.
  - Changes to `arg` affect the copy it referencs, not the original value

```
def main():
    value = 99
    print(f'The value is {value}.')
    change_me(value)
    print(f'Back in main the value is {value}.')


def change_me(arg):
    print('I am changing the value.')
    arg = 0
    print(f'Now the value is {arg}.')
```

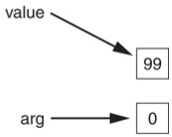


Figure 5: The value variable is not effected by changes to copy reference by arg (Gaddis, 2021, pg.265)

EAST TEXAS A&M

Keyword Arguments

- **Keyword argument**: argument that specifies which parameter the value should be passed to
  - Position when calling function is irrelevant
  - General format: `function_name(parameter=value)`
- Possible to mix keyword and positional arguments
  - Positional arguments must appear first

EAST TEXAS A&M

Global Variables and Global Constants

- **Global variable**: created by assignment statement written outside of any function
  - Can be accessed by any statement in the program file, including from within a function
  - If a function needs to assign a value to the global variable, the global variable must be redeclared within the function
    - General format: `global variable_name`

Global Variables Considered Bad (Mervin, 2017)

Reasons to avoid using global variables:

- Global variables make debugging difficult
  - Many locations in the code could be causing a wrong variable value
- Functions that use global variables are usually dependent on those variables
  - Makes function hard to transfer to another program
- Global variables make a program hard to understand

EAST TEXAS A&M

Global Constants

- **Global constant**: global name that references a value that cannot be changed
  - Permissible (and even good at times) to use global named constants in program
  - To simulate global constant in Python, create global variable and do not re-declare it within functions (don't use `global` keyword)

```
CONTRIBUTION_RATE = 0.05

def show_pay_contrib(gross):
    contrib = gross * CONTRIBUTION_RATE
    print(f'Contribution for gross pay: ${contrib:,.2f}.')

def show_bonus_contrib(bonus):
    contrib = bonus * CONTRIBUTION_RATE
    printf('Contribution for bonuses: ${contrib:,.2f}.')
```



Summary

- A **local variable** is a variable assigned a value inside of a function
  - Local variables are local in **scope** to the function where they are created
- **Arguments** are pieces of data that are sent as input into a function
  - Function can have 0, 1 or many arguments
  - Arguments are passed by value
  - Call a function by passing arguments in parentheses `()`,
    - Can use positional and/or keyword arguments when calling functions
- A variable declared outside of any function is **Global** in scope
  - Usually don't use global variables, but global named constants are acceptable



EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.

Mervin. (2017). *Why are global variables bad?* Retrieved November 11, 2017, from https://dev.to/mervinsv/why-global-variables-are-bad-4pj

EAST TEXAS A&M